

USE CLAUDE CODE

Manage sessions

Name, resume, branch, and switch between Claude Code conversations. Covers `--continue` , `--resume` , `--from-pr` , the `/resume` picker, session naming, exporting transcripts, and where transcripts are stored.

A session is a saved conversation tied to a project directory. Claude Code stores it locally as you work, so you can resume where you left off, branch to try a different approach, or switch between tasks.

The desktop app, Claude Code on the web, and the VS Code extension each maintain their own session history. This page covers the CLI.

Resume a session

Sessions are saved continuously to local transcript files as you work, so you can return to one after exiting or running `/clear` . Use these entry points:

Command	What it does
<code>claude --continue</code>	Resumes the most recent session in the current directory
<code>claude --resume</code>	Opens the <u>session picker</u>
<code>claude --resume <name></code>	Resumes the named session directly
<code>claude --from-pr <number></code>	Resumes the session linked to that pull request
<code>/resume</code>	Switches to a different conversation from inside an active session

Sessions created with `claude -p` or the Agent SDK do not appear in the session picker, but you can still resume one by passing its session ID to `claude --resume <session-id>` . Run this from the directory the session was started in: session ID lookup is scoped to the current project directory and its git worktrees, so a session created elsewhere reports `No conversation found with session ID: <session-id>` .

Where the session picker looks

Sessions are stored per project directory. By default the session picker shows interactive sessions from the current worktree, plus sessions started elsewhere that added the current directory with `/add-dir` . From v2.1.169, moving a session with `/cd` relocates it to the new directory’s project storage, so it appears in that directory’s picker afterward. Use `Ctrl+W` to widen to all worktrees of the repository or `Ctrl+A` to widen to every project on this machine.

Selecting a session from another worktree of the same repository resumes it in place. Selecting a session from an unrelated project copies a `cd` and resume command to your clipboard instead.

Resuming by name resolves across the current repository and its worktrees. Both forms look for an exact match and resume it directly even if it lives in a different worktree:

Command	Exact match	Ambiguous name
<code>claude --resume <name></code>	Resumes directly	Opens the session picker with the name pre-filled as a search term
<code>/resume <name></code>	Resumes directly	Reports an error; run <code>/resume</code> with no argument to open the session picker

Name your sessions

Give sessions descriptive names so they’re findable in the session picker and resumable by name. This matters most when you’re working on several tasks in parallel.

When	How to set the name
At startup	<code>claude -n auth-refactor</code>
During a session	<code>/rename auth-refactor</code> . The name also appears on the prompt bar
From the session picker	Highlight a session and press <code>Ctrl+R</code>
On plan accept	Accepting a plan in <u>plan mode</u> names the session from the plan content unless you’ve already set one

Once a session is named, return to it with `claude --resume <name>` or `/resume <name>` . See Resume a session for how name resolution behaves across worktrees.

Use the session picker

Run `/resume` inside a session, or `claude --resume` with no arguments, to open the interactive session picker. Use these keyboard shortcuts to navigate, search, and widen the list:

Shortcut	Action
<code>↑ / ↓</code>	Navigate between sessions
<code>→ / ←</code>	Expand or collapse grouped sessions
<code>Enter</code>	Resume the highlighted session
<code>Space</code>	Preview the session content. <code>Ctrl+V</code> also works on terminals that don't capture it as paste
<code>Ctrl+R</code>	Rename the highlighted session
<code>/</code> or any printable character other than <code>Space</code>	Enter search mode and filter sessions. Paste a GitHub, GitHub Enterprise, GitLab, or Bitbucket pull or merge request URL to find the session that created it
<code>Ctrl+A</code>	Show sessions from all projects on this machine. Press again to return to the current repository
<code>Ctrl+W</code>	Show sessions from all worktrees of the current repository. Press again to return to the current worktree. Only shown in multi-worktree repositories
<code>Ctrl+B</code>	Filter to sessions from the current git branch. Press again to show all branches
<code>Esc</code>	Exit the session picker or search mode

Each row shows the session name if set, otherwise the conversation summary or first prompt, along with time since last activity, message count, and git branch. Project path appears after you widen to all projects with `Ctrl+A`.

Forked sessions created with `/branch`, `/rewind`, or `--fork-session` are grouped under their root session. Press `→` to expand a group.

Branch a session

Branching creates a copy of the conversation so far and switches you into it, leaving the original intact. Use it to try a different approach without losing the path you were on.

From inside a session, run `/branch` with an optional name:

```
/branch try-streaming-approach
```

From the command line, combine `--continue` or `--resume` with `--fork-session` :

```
claude --continue --fork-session
```

The original session is unchanged and remains available in the session picker. The `/branch` confirmation prints two session IDs: the new branch you are now in and the original. To return to the original, pass its ID to `/resume` , use the session picker, or run `/resume <original-name>` . Permissions you approved with “allow for this session” do not carry over to the new branch. If you resume the same session in two terminals without forking, messages from both interleave into one transcript.

For checkpoint-based rewind within a single session, see [Checkpointing](#).

Manage context within a session

These commands control what’s in the context window without leaving the session:

- `/clear` : start fresh with an empty context. The previous conversation is saved and resumable
- `/compact [instructions]` : replace history with a summary, optionally focused on what you specify
- `/context` : show what is currently consuming context

For how compaction interacts with CLAUDE.md, skills, and rules, see the [context window guide](#). For strategies on when to clear versus compact, see [Best practices](#).

Export and locate session data

Run `/export` to copy the current conversation to your clipboard or save it as a plain-text file, with messages and tool outputs rendered as readable text. Pass a filename to write directly to that file.

Access conversations from scripts

`/export` produces a rendered transcript for a person to read. The interfaces below produce structured data for a script to parse: a JSON result from a run, the path to a session’s transcript file,

or a live stream of events. Pick by what triggers the script:

- **Run Claude once and capture the result:** invoke `claude -p` with `--output-format json` or `stream-json` to capture the result, session ID, usage, and cost of a non-interactive run as structured JSON.
- **Ask an existing session a question:** pass a session ID to `claude -p --resume` to send a follow-up prompt, such as a summary request, and capture the structured response.
- **React to session events:** read the `transcript_path` field that hooks and status line commands receive as input. A `SessionEnd` hook can archive the transcript when a session ends.
- **Embed Claude in a TypeScript or Python app:** use the Agent SDK to receive each message programmatically.

The example below uses the second interface. It sends a follow-up prompt to an existing session and reads the answer with `jq` :

```
claude -p --resume <session-id> --output-format json "summarize
what we changed" | jq -r '.result'
```

Where transcripts are stored

By default, transcripts are stored as JSONL at `~/.claude/projects/<project>/<session-id>.jsonl` , where `<project>` is your working directory path with non-alphanumeric characters replaced by `-` . Each line is a JSON object for a message, tool use, or metadata entry. The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release. To build on session data, use `/export` or the script interfaces instead.

The location, retention, and write behavior are configurable:

To	Set	Where
Move storage off <code>~/.claude</code>	<code>CLAUDE_CONFIG_DIR</code>	Environment variable
Change the 30-day retention	<code>cleanupPeriodDays</code>	<code>settings.json</code>
Suppress transcript writes in all modes	<code>CLAUDE_CODE_SKIP_PROMPT_HISTORY</code>	Environment variable
Suppress writes for one non-interactive run	<code>--no-session-persistence</code>	CLI flag with <code>claude -p</code>

See also

These pages cover related session and parallelism mechanics:

- **Worktrees**: run isolated parallel sessions on separate branches
- **Checkpointing**: rewind code and conversation to an earlier point
- **Context window**: what fills context and what survives compaction
- **Non-interactive mode**: session behavior under `claude -p`